

Tập luận văn đội tuyển quốc gia Trung Quốc năm 2017

Tập luận văn ứng viên đội tuyển quốc gia IOI Trung Quốc

Tháng 4 năm 2017

Nguồn gốc: Tập luận văn ứng viên đội tuyển quốc gia Trung Quốc Olympic Tin học năm 2017

IOI China candidate team papers / National training team 2017 collection.pdf

Tóm tắt

PDF nguồn là tập hợp luận văn đội tuyển quốc gia Trung Quốc năm 2017. Khác với một số tập trước đó, lớp văn bản của tập này trích xuất khá tốt bằng pdftotext. Bản LaTeX này chuyển ngữ phần nhận diện tập sách, toàn bộ mục lục, bài đầu tiên của Mao Xiao về công thức truy hồi của dãy số, và bài thứ hai của Yang Jiaqi về matching trên đồ thị tổng quát bằng đại số tuyến tính.

1 Thông tin đầu sách

Tập sách có tiêu đề *Tập luận văn ứng viên đội tuyển quốc gia Trung Quốc Olympic Tin học năm 2017*. Trang bìa ghi thời điểm phát hành là tháng 4 năm 2017. Tập PDF gồm 204 trang, được tạo bằng LaTeX với gói hyperref.

2 Mục lục đã dịch

Trang	Bài viết	Tác giả
1	Một số nghiên cứu về công thức truy hồi của dãy số	Mao Xiao
12	Matching trên đồ thị tổng quát dựa trên đại số tuyến tính	Yang Jiaqi
27	Tính tổng đa thức	Yuan Yutao
32	Bàn về bài toán tập độc lập trong thi tin học	Zhong Zhixian
50	Báo cáo ra đề và mở rộng của <i>Đồ thị con kỳ diệu</i>	Chen Junkun
75	Tìm hiểu bài toán bao đóng bắc cầu động	Sun Yaofeng
90	Báo cáo ra đề <i>A + B Problem</i>	Wang Leping
99	Bước đầu tìm hiểu thuật toán chia khối kích thước phi chuẩn	Xu Mingkuan
122	Cây palindrome và ứng dụng	Weng Wentao
139	Báo cáo ra đề <i>Cây đen trắng</i>	Yan Shuyi
144	Báo cáo ra đề <i>Đa giác đều</i>	Yang Jingqin
155	Bàn về lời giải tuyến tính cho quy hoạch động có đơn điệu quyết định	Feng Zhe
165	Báo cáo ra đề <i>Cây đoạn bị thao túng</i>	Shen Rui
179	Bước đầu về logic máy tính và nghệ thuật: mô hình cường độ nốt khi chơi piano dựa trên logic	Zhao Shengyu
190	Báo cáo ra đề <i>Tái cấu trúc bộ gen</i>	Hong Huadun

3 Một số nghiên cứu về công thức truy hồi của dãy số

Mao Xiao, Trường Trung học Yali

Tóm tắt

Bài viết này là luận văn đội tuyển quốc gia tin học Trung Quốc năm 2017 của tác giả. Trong bài, tác giả trước hết giới thiệu đa thức truy hồi, sau đó giới thiệu thuật toán Berlekamp–Massey, rồi trình bày triển vọng ứng dụng của chúng trong thi tin học, gồm đếm công thức truy hồi và tìm đa thức đặc trưng của một số ma trận thưa.

Dẫn nhập

Đa thức truy hồi là một thuật ngữ do tác giả đặt ra. Trong thi tin học, công thức truy hồi thường được cho tường minh, còn nhiệm vụ của thí sinh thường là: biết một số hạng đầu của dãy và một công thức truy hồi nào đó, hãy tính một số hạng của dãy. Tuy nhiên, bài viết này nghiên cứu những công thức truy hồi ẩn. Vì vậy tác giả đưa ra khái niệm hoàn toàn mới là đa thức truy hồi. Đây là một khái niệm rất mạnh: nó không chỉ có các ứng dụng như đếm công thức truy hồi, mà còn giúp ta học và hiểu thuật toán Berlekamp–Massey.

Berlekamp–Massey là một thuật toán xuất sắc nhưng bị lạnh nhạt. Trong thi tin học, Berlekamp–Massey không mấy phổ biến; ngay cả những người biết nó cũng thường chỉ xem nó như một đường tắt. Ngay cả trên phạm vi thế giới, tôi cũng chưa nghe nói thuật toán này từng xuất hiện như thuật toán chuẩn của một bài nào. Tuy vậy, nó có khá nhiều ứng dụng.

Do đó, bài viết này sẽ trình bày hai nội dung ấy và giới thiệu một số ứng dụng của chúng.

3.1 Đa thức truy hồi

Trong mục này, tác giả sẽ đưa ra nhiều định nghĩa mới, đồng thời giới thiệu nhiều kết luận hữu ích.

3.1.1 Bậc nhỏ nhất

Định nghĩa 1.1. Với dãy $\{s_0, s_1, \dots, s_{l-1}\}$, ta đặt đa thức tương ứng của dãy là

$$S(x) = \sum_{k=0}^{l-1} s_k x^k.$$

Định nghĩa 1.2. Với một đa thức khác không A , **bậc** của nó được định nghĩa là bậc của hạng tử cao nhất, ký hiệu $\deg(A)$. Riêng đa thức không có bậc là vô cùng nhỏ.

Định nghĩa 1.3. Trên một trường nào đó, với dãy $\{r_0, r_1, \dots, r_{m-1}\}$ và dãy $\{a_0, a_1, \dots, a_{n-1}\}$, nếu r không toàn bằng không và với mọi $0 \leq p \leq n - m$ ta có

$$\sum_{k=0}^{m-1} a_{p+k} \times r_{m-k-1} = 0, \quad (1.1)$$

thì gọi r là một **công thức truy hồi**¹ của dãy a .

Nếu $r_0 = 1$, ta gọi r là **công thức truy tiến**. Nếu r là công thức truy tiến có độ dài ngắn nhất trong mọi công thức truy tiến của dãy a , ta gọi nó là **công thức truy tiến ngắn nhất** của dãy a .

¹Một số nơi gọi khái niệm này là công thức truy tiến. Tác giả không tìm được khác biệt giữa hai thuật ngữ, nên trong bài chọn thuật ngữ ít dùng hơn để định nghĩa riêng, còn thuật ngữ kia giữ nghĩa thường gặp.

Định nghĩa 1.4. Với dãy a và đa thức khác không $R(x)$, luôn tồn tại duy nhất công thức truy hồi ngắn nhất r sao cho đa thức tương ứng của nó là $R(x)$. Ta gọi r là **công thức truy hồi ngắn nhất tương ứng** của $R(x)$. Độ dài của r trừ một được gọi là **bậc nhỏ nhất** của đa thức $R(x)$, ký hiệu d_R hoặc $d_{R(x)}$. Đa thức $R(x)$ được gọi là **đa thức truy hồi bậc** d_R của a .

Chứng minh. Cho một đa thức khác không $R(x)$ và một dãy a độ dài n . Dãy gồm các hệ số từ hạng tử bậc 0 đến hạng tử bậc $\max(\deg(R), n)$ của nó chắc chắn là một công thức truy hồi của a , và đa thức tương ứng là $R(x)$, nên r tồn tại.

Nếu bậc nhỏ nhất của đa thức khác không $R(x)$ là d , thì r nhất định là dãy gồm các hệ số từ bậc 0 đến bậc d của nó, nên r là duy nhất. ■

3.1.2 Các phép toán trên đa thức truy hồi

Sau khi có đa thức truy hồi, ta có thể nhận được một số kết luận hữu ích. Từ các kết luận này, ta có thể làm nhiều việc, và chúng cũng giúp ích cho thuật toán Berlekamp–Massey.

Bổ đề 1.1. Đa thức $R(x)$ có bậc nhỏ nhất d đối với dãy a khi và chỉ khi d không nhỏ hơn bậc của $R(x)$, và các hệ số từ bậc d đến bậc $n - 1$ của $R(x) \times A(x)$ đều bằng không.

Chứng minh. Ta nhận thấy công thức (1.1) có dạng tích chập; từ đó dễ suy ra bổ đề. ■

Định lý 1.1. Giả sử $F(x), G(x)$ là hai đa thức có bậc nhỏ nhất không vượt quá d đối với dãy a . Khi đó $F(x) + G(x)$ cũng có bậc nhỏ nhất không vượt quá d đối với dãy a .

Chứng minh. Theo Bổ đề 1.1, các hệ số từ bậc d đến bậc $n - 1$ của $F(x) \times A(x)$ và $G(x) \times A(x)$ đều bằng không. Vì vậy các hệ số từ bậc d đến bậc $n - 1$ của $(F(x) + G(x)) \times A(x)$ đều bằng không; đồng thời bậc của $F(x) + G(x)$ hiển nhiên không vượt quá d . Do đó $F(x) + G(x)$ có bậc nhỏ nhất không vượt quá d đối với dãy a . ■

Bổ đề 1.2. Giả sử $F(x)$ có bậc nhỏ nhất d đối với dãy a . Khi đó với $k \in \mathbb{N}$ và $k \geq 0$, $x^k F(x)$ có bậc nhỏ nhất không vượt quá $d + k$ đối với dãy a .

Chứng minh. Theo Bổ đề 1.1, các hệ số từ bậc d đến bậc $n - 1$ của $F(x) \times A(x)$ đều bằng không. Vì vậy các hệ số từ bậc $d + k$ đến bậc $n + k - 1$ của $x^k F(x) \times A(x)$ đều bằng không; đồng thời bậc của $x^k F(x) \times A(x)$ hiển nhiên không vượt quá $d + k$. Do đó $x^k F(x)$ có bậc nhỏ nhất không vượt quá $d + k$ đối với dãy a . ■

Định lý 1.2. Giả sử $F(x)$ có bậc nhỏ nhất d_F đối với dãy a . Khi đó $F(x) \times G(x)$ có bậc nhỏ nhất không vượt quá $d_F + \deg(G)$ đối với dãy a .

Chứng minh. Suy ra từ Định lý 1.1 và Bổ đề 1.2. ■

3.1.3 Cơ sở

Định nghĩa 1.5. Với dãy a và dãy đa thức $\{\text{Base}_0(x), \text{Base}_1(x), \text{Base}_2(x), \dots\}$, nếu với mọi k , $\text{Base}_k(x)$ là đa thức có bậc nhỏ nhất nhỏ nhất trong mọi đa thức có hạng tử thấp nhất là x^k , thì gọi dãy đa thức Base là **một cơ sở** của dãy a .

Ta nhận thấy một cơ sở của một dãy thật ra chính là cơ sở của không gian tuyến tính do nó sinh ra. Ta cũng có thể nhận thấy tọa độ của một đa thức theo một cơ sở liên hệ chặt chẽ với bậc nhỏ nhất của nó.

Định lý 1.3. Nếu Base là cơ sở của dãy a , thì mọi đa thức có bậc nhỏ nhất d đều có thể được biểu diễn tuyến tính một cách duy nhất bởi một số $\text{Base}_k(x)$ có bậc nhỏ nhất không vượt quá d ; hơn nữa trong biểu diễn đó nhất định tồn tại ít nhất một $\text{Base}_k(x)$ có bậc nhỏ nhất đúng bằng d .

Chứng minh. Giả sử $R(x)$ là một đa thức có bậc nhỏ nhất d . Ta xét cách phân rã nó.

Nếu $R(x)$ bằng không thì không cần phân rã. Ngược lại, giả sử hạng tử thấp nhất của nó là x^k , hệ số là C_r , và hệ số tương ứng của $\text{Base}_k(x)$ là C_{Base} . Khi đó hạng tử thấp nhất của $R(x) - C_{\text{Base}}^{-1} C_r \text{Base}_k(x)$ ít nhất là x^{k+1} . Vì $\text{Base}_k(x)$ là một đa thức có bậc nhỏ nhất nhỏ nhất trong các đa thức có hạng tử thấp nhất x^k , bậc nhỏ nhất của nó hiển nhiên không vượt quá d . Theo Định lý 1.1, $R(x) - C_{\text{Base}}^{-1} C_r \text{Base}_k(x)$ cũng có bậc nhỏ nhất không vượt quá d . Ta phân rã biểu thức này theo cách đệ quy. Cuối cùng ta thu được một biểu diễn tuyến tính thỏa điều kiện.

Vì các $\text{Base}_k(x)$ là một cơ sở của không gian tuyến tính mà chúng sinh ra, cách biểu diễn hiển nhiên là duy nhất.

Nếu trong biểu diễn, mọi $\text{Base}_k(x)$ đều có bậc nhỏ nhất nhỏ hơn d , thì hoặc đa thức này là đa thức không, hoặc theo Bổ đề 1.1 bậc nhỏ nhất của nó sẽ nhỏ hơn d , mâu thuẫn. ■

3.2 Thuật toán Berlekamp–Massey

Mục này giới thiệu thuật toán Berlekamp–Massey². Tuy các tài liệu như Wikipedia [1] đều đã giới thiệu quy trình của thuật toán Berlekamp–Massey, tác giả không tìm được bất kỳ bản quy trình thuật toán tiếng Trung nào. Đồng thời, tác giả cũng muốn dùng ngôn ngữ của bài viết này để trình bày lại quy trình của thuật toán.

3.2.1 Giới thiệu thuật toán

Thuật toán Berlekamp–Massey có thể tìm một cơ sở Base cho dãy a độ dài n trên một trường bất kỳ bằng $\mathcal{O}(n^2)$ phép toán và không cần thêm không gian đáng kể.

3.2.2 Quy trình thuật toán

Theo định nghĩa, mọi đa thức có hạng tử thấp nhất x^k , với $k \geq n$, đều có bậc nhỏ nhất đúng bằng bậc của nó. Vì vậy với $\text{Base}_k(x)$, ta chỉ cần chọn bất kỳ Cx^k nào với $C \neq 0$. Không mất tính tổng quát, giả sử mọi đa thức tìm được đều có hệ số của hạng tử thấp nhất bằng 1, khi đó ở đây ta đặt $C = 1$.

Giả sử ta đã tìm được một đa thức có bậc nhỏ nhất nhỏ nhất trong các đa thức từ $\text{Base}_{k+1}(x)$ đến $\text{Base}_n(x)$. Xét cách tìm $\text{Base}_k(x)$.

Ta xét $x \text{Base}_k(x)$. Theo Định lý 1.3, ta chỉ cần làm cho bậc nhỏ nhất lớn nhất trong biểu diễn tuyến tính của nó càng nhỏ càng tốt.

²Wikipedia tiếng Trung phiên âm thuật toán này là Berlekamp–Massey.

Trước hết, vì hạng tử thấp nhất của $x \text{Base}_k(x)$ là x^{k+1} , trong biểu diễn nhất định có hạng $\text{Base}_{k+1}(x)$. Xét các hạng còn lại: rõ ràng bậc nhỏ nhất lớn nhất trong số chúng phải càng nhỏ càng tốt.

Đặt $\text{res}(x) = x \text{Base}_k(x) - \text{Base}_{k+1}(x)$. Ta tính $\text{pro}(x) = \text{res}(x) \times A(x)$. Theo Bổ đề 1.1, để biết các hệ số từ bậc $k+1$ đến bậc $n-1$ của $\text{pro}(x)$ đều bằng không.

Xét hạng tử bậc n . Nếu nó bằng không, ta đã biết $x \text{Base}_k(x)$; ngược lại giả sử nó là u . Xét $\text{Base}_l(x)$ với $l > k+1$. Nếu hệ số bậc n của $\text{Base}_l(x) \times A(x)$ khác không, giả sử hệ số bậc n là v . Để biết các hệ số từ bậc $k+1$ đến bậc n của $\text{res}(x) - uv^{-1} \text{Base}_l(x)$ đều bằng không. Vì bậc nhỏ nhất lớn nhất phải càng nhỏ càng tốt, nếu tồn tại đa thức thỏa điều kiện thì $\text{Base}_l(x)$ nhất định là đa thức có bậc nhỏ nhất nhỏ nhất trong mọi đa thức thỏa điều kiện. Lấy $\text{Base}_l(x)$ như vậy, ta có thể tìm được $x \text{Base}_k(x)$. Nếu không tồn tại $\text{Base}_l(x)$ như vậy, thì chỉ có thể là bậc nhỏ nhất của $x \text{Base}_k(x)$ đã vượt quá n . Khi đó ta chỉ cần tùy ý chọn một đa thức có hạng tử thấp nhất là hạng bậc x^{k+1} và hệ số bằng 1.

Khi đã biết $x \text{Base}_k(x)$, hiển nhiên ta cũng biết $\text{Base}_k(x)$.

Dùng phương pháp này, mỗi $P_k(x)$ đều có thể được tìm trong không quá $\mathcal{O}(n)$ phép toán, vì vậy tổng số phép toán là $\mathcal{O}(n)$, còn số Base cần tính chỉ là $\mathcal{O}(n)$. Do đó tổng số phép toán là $\mathcal{O}(n^2)$.

Rõ ràng ngoài việc lưu $A(x)$ và Base, thuật toán này không cần không gian phụ đáng kể.

3.2.3 Mã giả

Theo mô tả trên, thuật toán có thể được cài đặt bằng phương pháp sau.

Thuật toán 1 Berlekamp–Massey Algorithm I

Require: $a, n = |a|$

Ensure: Base

```

1:  $\text{Base}_k = x^k$  ( $k \geq n$ )
2:  $l \leftarrow n + 1$ 
3:  $v \leftarrow 1$ 
4: for  $k \leftarrow n - 1$  to 1 do
5:    $u \leftarrow [x^n] \text{Base}_{k+1}(x) \times A(x)$ 
6:    $\text{Base}_k(x) \leftarrow (\text{Base}_{k+1}(x) - uv^{-1} \text{Base}_l(x)) \div x$ 
7:   if  $(u \neq 0) \wedge (d_{\text{Base}_{k+1}} < d_{\text{Base}_l})$  then
8:      $l \leftarrow k + 1$ 
9:      $v \leftarrow u$ 
10:  end if
11: end for
```

Theo mô tả thuật toán, giá trị ban đầu của l và v không quan trọng; chỉ cần giá trị ban đầu thỏa $l \geq n+1$ là có thể bảo đảm đáp án đúng. Tuy nhiên, $l = n+1, v = 1$ là giá trị khởi tạo tham khảo trên Wikipedia tiếng Anh, nên tác giả xem nó như một phương án đã thành thông lệ.

Tuy nhiên, vì phép toán đa thức khá phiền phức, khi cài đặt thật sự ta thường không dùng đa thức một cách tường minh.

Ký hiệu r_k là công thức truy hồi ngắn nhất tương ứng với $\text{Base}_{n-k+1} \div x^{n-k+1}$. Khi đó ta dễ thấy r_k là công thức truy tiến ngắn nhất của tiền tố độ dài k của dãy a . Vì vậy quá trình trên dễ dàng được đổi thành quá trình tính r .

Ta xem dãy như một mảng có độ dài thay đổi, chỉ số bắt đầu từ 0. Thuật toán mới như sau.

Thuật toán 2 Berlekamp–Massey Algorithm II

Require: $a, n = |a|$

Ensure: r_k ($0 \leq k \leq n$)

```
1:  $r_0 = \{1\}$ 
2:  $\text{last} \leftarrow \{1\}$ 
3:  $v \leftarrow 1$ 
4:  $s \leftarrow 1$ 
5: for  $k \leftarrow 1$  to  $n$  do
6:    $r_k \leftarrow r_{k-1}$ 
7:    $u \leftarrow \sum_{i=0}^{|r_k|-1} a_{k-i-1} \times r_i$ 
8:   if ( $u \neq 0$ ) then
9:     if  $|r_k| < |\text{last}| + s$  then
10:       $r_k.\text{append}(\{0\} \times (|\text{last}| + s - |r_k|))$ 
11:      for  $i \leftarrow 0$  to  $|\text{last}| - 1$  do
12:         $r_k[i + s] \leftarrow r_k[i + s] - uv^{-1} \text{last}[i]$ 
13:      end for
14:       $\text{last} \leftarrow r_{k+1}$ 
15:       $v \leftarrow u$ 
16:       $s \leftarrow 0$ 
17:    else
18:      for  $i \leftarrow 0$  to  $|\text{last}| - 1$  do
19:         $r_k[i + s] \leftarrow r_k[i + s] - uv^{-1} \text{last}[i]$ 
20:      end for
21:    end if
22:  end if
23:   $s \leftarrow s + 1$ 
24: end for
```

Ngoài ra, nếu ta chỉ cần tìm công thức truy tiến ngắn nhất của cả dãy, thì ở mọi thời điểm ta thật ra chỉ cần lưu một r_k và một last . Nếu không tính kích thước từng phần tử, độ phức tạp không gian có thể giảm xuống $\mathcal{O}(n)$.

3.3 Đếm công thức truy hồi

3.3.1 Đa thức truy hồi nhỏ nhất và đa thức truy hồi thứ nhì không tầm thường

Định nghĩa 3.1. Trong mọi $\text{Base}_k(x)$ đã tìm được, một đa thức có bậc nhỏ nhất nhỏ nhất được gọi là **đa thức truy hồi nhỏ nhất**, ký hiệu $M(x)$. Nếu có nhiều đa thức như vậy thì chọn tùy ý; bậc nhỏ nhất của nó được ký hiệu d_M .

Trong mọi $\text{Base}_k(x)$ đã tìm được, nếu tồn tại một đa thức không thể biểu diễn dưới dạng $x^k M(x)$, với $k \in \mathbb{N}, k \geq 0$, thì đa thức có bậc nhỏ nhất nhỏ nhất trong số đó được gọi là **đa thức truy hồi thứ nhì không tầm thường**, ký hiệu $S(x)$. Nếu có nhiều đa thức như vậy thì chọn tùy ý; nếu không tồn tại thì đặt $S(x) = \text{Base}_{n+1}(x)$. Bậc nhỏ nhất của nó được ký hiệu d_S .

3.3.2 Biểu diễn tuyến tính

Định nghĩa 3.2. Với các đa thức $A(x), B(x), C(x)$, nếu tồn tại duy nhất đa thức $F_B(x)$ có bậc không vượt quá $d_A - d_B$ và đa thức $F_C(x)$ có bậc không vượt quá $d_A - d_C$ sao cho

$$A(x) = F_B(x)B(x) + F_C(x)C(x),$$

thì gọi $A(x)$ là **có thể biểu diễn tuyến tính** bởi $B(x), C(x)$.

Bổ đề 3.1. Với các đa thức $A(x), B(x), C(x), D(x), E(x)$, nếu $A(x)$ có thể biểu diễn tuyến tính bởi $B(x), C(x)$, và $B(x), C(x)$ đều có thể biểu diễn tuyến tính bởi $D(x), E(x)$, thì $A(x)$ cũng có thể biểu diễn tuyến tính bởi $D(x), E(x)$.

Chứng minh. Đặt

$$\begin{aligned} A(x) &= F_B(x)B(x) + F_C(x)C(x), \\ B(x) &= F_{BD}(x)D(x) + F_{BE}(x)E(x), \\ C(x) &= F_{CD}(x)D(x) + F_{CE}(x)E(x). \end{aligned}$$

Dễ biết

$$\begin{aligned} A(x) &= (F_B(x)F_{BD}(x) + F_C(x)F_{CD}(x))D(x) \\ &\quad + (F_D(x)F_{BE}(x) + F_C(x)F_{CE}(x))E(x). \end{aligned}$$

Vì

$$\deg(F_B(x)) \leq d_A - d_B, \quad \deg(F_{BD}(x)) \leq d_B - d_D,$$

nên

$$\deg(F_B(x)F_{BD}(x)) = \deg(F_B(x)) + \deg(F_{BD}(x)) \leq d_A - d_D.$$

Tương tự,

$$\deg(F_C(x)F_{CD}(x)) \leq d_A - d_D.$$

Do đó

$$\deg(F_B(x)F_{BD}(x) + F_C(x)F_{CD}(x)) \leq d_A - d_D.$$

Tương tự,

$$\deg(F_B(x)F_{BE}(x) + F_C(x)F_{CE}(x)) \leq d_A - d_E.$$

Quá trình này bảo đảm tính duy nhất, nên bổ đề được chứng minh. ■

Định lý 3.1. Sau khi tính xong $\text{Base}_k(x)$, nó là đa thức nhỏ nhất hoặc là đa thức thứ nhì không tầm thường.

Tạm thời ta chưa chứng minh định lý này; trước hết giới thiệu một bổ đề mới, rồi chứng minh cả hai cùng lúc.

Bổ đề 3.2. Mọi $\text{Base}_k(x)$ đều có thể được biểu diễn tuyến tính bởi $S(x)$ và $M(x)$.

Chứng minh. Xét quy trình của thuật toán Berlekamp–Massey.

Khi $k = n$, hiển nhiên Định lý 3.1 và Bổ đề 3.2 đều đúng.

Giả sử sau khi tính xong $\text{Base}_{k+1}(x)$, cả hai đều đúng.

Xét quá trình tính $x \text{Base}_k(x)$. Cuối cùng nó nhất định là $\text{Base}_{k+1}(x)$ cộng với một hằng số nhân với một $\text{Base}_l(x)$ nào đó. Theo mô tả thuật toán, ta dễ nhận thấy $\text{Base}_{k+1}(x)$ và hạng còn lại đều là một trong $M(x)$ và $S(x)$.

Vì vậy bậc nhỏ nhất của $x \text{Base}_k(x)$ bằng $\max(d_M, d_S)$, còn bậc nhỏ nhất của $\text{Base}_k(x)$ phải trừ thêm một. Do đó nó nhất định là đa thức nhỏ nhất hoặc đa thức thứ nhì tầm thường. Như vậy ta đã chứng minh Định lý 3.1.

Hiển nhiên với chính nó, Bổ đề 3.2 đúng. Tuy nhiên vì $M(x), S(x)$ đã thay đổi, ta còn phải xét $\text{Base}_l(x)$ với $l > k$.

Xét sự thay đổi của $M(x)$ và $S(x)$. $M(x)$ hoặc không đổi, hoặc trở thành $S(x)$ mới.

Xét $S(x)$. Theo quy trình thuật toán, ta có

$$x \text{Base}_k(x) = C_M M(x) + C_S S(x).$$

Do đó

$$S(x) = x \text{Base}_k(x) - C_M M(x).$$

Vì $d_S > d_{\text{Base}_k}$ và $d_S \geq d_M$, $S(x)$ trước khi thay đổi có thể được biểu diễn tuyến tính bởi $M(x)$ và $S(x)$ sau khi thay đổi.

Như vậy, $M(x)$ và $S(x)$ trước khi thay đổi đều có thể được biểu diễn tuyến tính bởi $M(x)$ và $S(x)$ sau khi thay đổi. Theo Bổ đề 3.1, $\text{Base}_l(x)$ với $l > k$ vẫn có thể được biểu diễn tuyến tính bởi $S(x)$ và $M(x)$.

Đồng thời, với trường hợp $k > n$, vì $\text{Base}_k(x) = x^{k-n} \text{Base}_n(x)$, bổ đề hiển nhiên đúng.

Tương tự, quá trình này giữ tính duy nhất, nên bổ đề được chứng minh. ■

Định lý 3.2. Mọi đa thức đều có thể được biểu diễn tuyến tính bởi $S(x)$ và $M(x)$.

Chứng minh. Dễ suy ra từ Định lý 1.3 và Bổ đề 3.2. ■

3.3.3 Định lý đếm công thức truy hồi

Định lý 3.3 (Định lý đếm đa thức truy hồi). Giả sử kích thước của trường là q . Khi đó số đa thức truy hồi bậc k của một dãy là

$$\text{num}_k = q^{\max(k-d_M-1,0)+\max(k-d_S-1,0)} (q^{[k \geq d_M]+[k \geq d_S]} - 1).$$

Trong công thức này, $[x]$ bằng 1 khi x đúng, và bằng 0 nếu không.

Chứng minh. Dễ suy ra từ Định lý 3.2. ■

Định lý 3.4 (Định lý đếm công thức truy hồi). Số công thức truy hồi của một dãy là

$$\sum_{k=1}^n (n - k + 1) \text{num}_k.$$

Chứng minh. Một đa thức truy hồi bậc x tương ứng đúng một công thức truy hồi có độ dài $x, x+1, \dots, n$, tổng cộng $n - x + 1$ công thức, nên có công thức trên. ■

3.4 Đa thức đặc trưng của ma trận thưa đặc biệt

Thuật toán Berlekamp–Massey không chỉ có tác dụng rất mạnh trong lĩnh vực công thức truy hồi; nó cũng có thể giúp ích cho các bài toán liên quan đến ma trận.

Vì vậy, chương này sẽ giới thiệu cách dùng thuật toán đó để tính đa thức đặc trưng của ma trận thưa đặc biệt.

3.4.1 Điều kiện quan trọng

Khi mỗi trị riêng chỉ tương ứng với một khối Jordan³, đa thức đặc trưng chính là đa thức tối tiểu. Đây là điều kiện quan trọng để thuật toán này hoạt động.

³Nói cách khác, chiều cao của mỗi vector đều bằng bội số của nó.

3.4.2 Quá trình thực hiện

Theo định lý Cayley–Hamilton, đa thức đặc trưng của một ma trận $n \times n$ A là đa thức triệt tiêu của nó; tức là nếu đa thức đặc trưng là p thì $p(A) = 0$.

Nhớ lại định nghĩa công thức truy hồi, ta thấy nếu chọn ngẫu nhiên một vector v , trước hết hiển nhiên $p(Av) = 0$. Nếu kích thước của trường số đủ lớn, thì với xác suất rất cao, dãy vector $\{v, Av, A^2v, \dots\}$ tồn tại một công thức truy tiến ngắn nhất duy nhất r có độ dài n . Khi đó $R(x)$ chính là đa thức đặc trưng.

Nếu A là ma trận thưa và số phần tử là e , thì từ $A^k v$ có thể tính $A^{k+1}v$ trong $\mathcal{O}(e)$ phép toán.

Khi xét dùng thuật toán Berlekamp–Massey, ta có hai khó khăn.

Khó khăn thứ nhất là xử lý nhanh dãy vector. Ta có thể thiết kế một hàm băm, trong đó hàm băm là một biến đổi tuyến tính, biến mọi vector thành một số; như vậy ta có thể xử lý dãy vector.

Khó khăn thứ hai là xử lý một dãy vô hạn. Tuy dãy này dài vô hạn, ta có thể nhận thấy trong trường hợp ngẫu nhiên, độ dài công thức truy tiến ngắn nhất của tiền tố độ dài $2n$ xấp xỉ n . Vì vậy ta có thể lấy một tiền tố đủ dài để tính. Một cách tốt hơn là xét Thuật toán 2: thật ra ta có thể liên tục tăng tiền tố cho đến khi độ dài công thức truy tiến ngắn nhất đạt n .

Như vậy, ta thu được một thuật toán Monte Carlo có thể tính đa thức đặc trưng của ma trận thưa đặc biệt bằng $\mathcal{O}(n(n + e))$ phép toán và không gian tuyến tính theo kích thước ma trận.

3.4.3 Định thức của ma trận thưa

Xét cách tính định thức $\det(A)$. Hiển nhiên nếu ma trận này thỏa điều kiện quan trọng ở Mục 4.1, ta có thể tìm đa thức đặc trưng, rồi lấy hạng tử tự do nhân với $(-1)^n$.

Nếu ma trận này không thỏa điều kiện đó, ta có thể lấy ngẫu nhiên một ma trận đường chéo B ; số phép toán là $\mathcal{O}(ne)$. Nếu trường số đủ lớn, thì AB có xác suất rất cao thỏa điều kiện này. Vì $\det(AB) = \det(A) \det(B)$, ta chỉ cần tính $\det(AB)$, rồi chia cho $\det(B)$. Vì B là ma trận đường chéo, $\det(B)$ chính là tích các phần tử trên đường chéo của B .

Như vậy, ta thu được một thuật toán Monte Carlo có thể tính định thức của ma trận thưa bằng $\mathcal{O}(n(n + e))$ phép toán và không gian tuyến tính theo kích thước ma trận.

3.4.4 Đếm cây khung của đồ thị thưa

Hiển nhiên, có thể dùng định lý ma trận-cây Kirchhoff để tính số cây khung. Nếu đồ thị đủ thưa, thì định thức của ma trận Laplace hiển nhiên có thể được tính bằng phương pháp ở Mục 4.3.

Độ phức tạp thời gian của thuật toán này là $\mathcal{O}(n(n + m))$, độ phức tạp không gian là $\mathcal{O}(n + m)$, rất tốt.

3.5 Tổng kết

Toàn bài đều xoay quanh công thức truy hồi. Phần thứ nhất trình bày đa thức truy hồi, phần thứ hai nghiên cứu thuật toán Berlekamp–Massey, còn phần thứ ba và thứ tư chủ yếu giới thiệu một số ứng dụng của chúng.

Tôi hy vọng bài viết này có thể đóng vai trò gợi mở. Đây vẫn chỉ là một nghiên cứu chưa chín muồi về công thức truy hồi, và các ứng dụng được giới thiệu cũng khá cơ bản. Khi khoa học kỹ thuật không ngừng tiến bộ, nhất định còn nhiều ứng dụng thú vị hơn đang chờ chúng ta khám phá. Vì vậy, mong rằng bài viết này có thể gợi ý cho độc giả và dẫn ra một số ứng dụng hấp dẫn hơn.

Lời cảm ơn

1. Cảm ơn bạn Du Yuhao. Lần đầu tiên tôi nghe nói về thuật toán này là từ bạn ấy, và sự xuất hiện của một phần nội dung trong bài viết này không thể tách rời sự giúp đỡ của bạn ấy.
2. Cảm ơn các bạn Du Yuhao, Yang Jiaqi và Weng Wentao đã sửa lỗi.
3. Cảm ơn China Computer Federation đã cung cấp nền tảng học tập và trao đổi.

Tài liệu tham khảo

[1] Wikipedia.

https://en.wikipedia.org/wiki/Berlekamp%E2%80%93Massey_algorithm

4 Matching trên đồ thị tổng quát dựa trên đại số tuyến tính

Yang Jiaqi, Trường Trung học Tưởng niệm Trung Sơn, thành phố Trung Sơn

Tóm tắt

Bài toán matching trên đồ thị là một bài toán kinh điển của lý thuyết đồ thị, có vị trí quan trọng cả trong học thuật lẫn trong thi thuật toán. Bài viết này chủ yếu giới thiệu một thuật toán matching trên đồ thị tổng quát dựa trên đại số tuyến tính, đồng thời đưa ra một số ứng dụng đơn giản của nó. Thuật toán này rất khác với các phương pháp tổ hợp truyền thống để giải bài toán này; hy vọng bản thân thuật toán và những tư tưởng dùng trong quá trình thu được thuật toán có thể gợi mở cho độc giả.

Dẫn nhập

Bài toán matching trên đồ thị có thể được chia theo tính chất của đồ thị thành hai loại: matching trên đồ thị hai phía và matching trên đồ thị tổng quát. Trong đó, matching hai phía đã xuất hiện nhiều năm trong giới thi thuật toán và là một điểm kiến thức thường gặp. Vì thuật toán của nó tương đối đơn giản, nó đã được phổ cập rất tốt: hiện nay phần lớn thí sinh đều đã nắm vững và có thể vận dụng thành thạo thuật toán matching hai phía.

Ngược lại, matching trên đồ thị tổng quát là một điểm kiến thức mới nổi. Vài năm gần đây, nó đã xuất hiện trong bài tập đội tuyển, kỳ thi mùa đông và các kỳ thi giao lưu đội tuyển. Nhưng các kỳ thi ấy đều ở cấp trên NOI; trong các kỳ thi dưới NOI, đề về matching tổng quát còn hiếm gặp, và hiện nay số thí sinh nắm được thuật toán matching tổng quát cũng tương đối ít. Tác giả cho rằng một nguyên nhân quan trọng khiến thuật toán này chưa phổ biến là thuật toán cây hoa, cách thường dùng hiện nay để giải loại bài toán này, khá phức tạp và có ngưỡng học tập cao. Vì vậy bài viết này sẽ giới thiệu một thuật toán matching dễ nhớ, dễ cài đặt, với hy vọng thúc đẩy việc phổ cập thuật toán matching trên đồ thị tổng quát.

Như tiêu đề bài viết đã nói, thuật toán được giới thiệu ở đây là một phương pháp đại số. Dù trong thi thuật toán, phương pháp tổ hợp thường gặp hơn, phương pháp đại số cũng là một công cụ mạnh để giải các bài toán đồ thị. Ví dụ nổi tiếng nhất có lẽ là định lý Matrix-Tree để đếm cây khung. Phương pháp đại số đem lại một góc nhìn hoàn toàn mới khi giải quyết vấn đề; hy vọng bài viết này có thể giúp độc giả cảm nhận sức hấp dẫn riêng của phương pháp đại số.

Bài viết trước hết sẽ bắt đầu từ bài toán matching hoàn hảo, giải bài toán quyết định tương ứng, sau đó thu được một thuật toán xây dựng phương án matching hoàn hảo và từng bước tối ưu nó bằng một số kiến thức đại số tuyến tính. Tiếp theo, bài viết chuyển bài toán matching cực đại về matching hoàn hảo, rồi cuối cùng trình bày một số ứng dụng và mở rộng của thuật toán này.

4.1 Kiến thức chuẩn bị

4.1.1 Lý thuyết đồ thị

Nếu không nói gì thêm, các đồ thị trong bài đều là đồ thị vô hướng.

Ta dùng $G = (V, E)$ để biểu diễn một đồ thị, trong đó V là tập đỉnh và E là tập cạnh. Ta thường dùng n để biểu diễn kích thước $|V|$ của tập đỉnh, dùng m để biểu diễn kích thước $|E|$ của tập cạnh, và thường xem $V = \{v_1, v_2, \dots, v_n\}$.

Khi không gây nhập nhằng, ta dùng uv để biểu diễn cạnh (u, v) .

Một **matching** M của đồ thị $G = (V, E)$ là một tập con của tập cạnh E , thỏa mãn hai cạnh bất kỳ trong M không có đỉnh chung. Nếu đỉnh v là đầu mút của một cạnh nào đó trong M , ta nói v nằm trong matching M . Giá trị lớn nhất của kích thước mọi matching của đồ thị G được ký hiệu là $\nu(G)$; nếu $|M| = \nu(G)$, ta gọi M là một **matching cực đại** của G . Đặc biệt, nếu mọi đỉnh của G đều nằm trong matching M , ta gọi M là một **matching hoàn hảo** của G . Rõ ràng G chỉ có thể có matching hoàn hảo khi $|V|$ là số chẵn.

Với một đồ thị $G = (V, E)$ và một tập đỉnh $U \subseteq V$, ta ký hiệu đồ thị thu được bằng cách xóa mọi đỉnh trong U khỏi G là $G - U$, tức là

$$G - U = (V \setminus U, E \cap \{uv \mid u, v \in V \setminus U\});$$

đồ thị con cảm sinh của G theo U được ký hiệu là $G[U]$, tức là

$$G[U] = G - (V \setminus U).$$

4.1.2 Đại số tuyến tính

Với một ma trận A có m hàng và n cột, ta đặt tập chỉ số hàng là $\{1, \dots, m\}$, tập chỉ số cột là $\{1, \dots, n\}$. Phần tử ở hàng i , cột j của ma trận A được ký hiệu là $A_{i,j}$.

Với một ma trận $m \times n$ A và các tập bất kỳ $I \subseteq \{1, \dots, m\}, J \subseteq \{1, \dots, n\}$, ta ký hiệu ma trận con thu được bằng cách giữ lại mọi hàng trong I và mọi cột trong J là $A_{I,J}$. Ta viết tắt $A_{I,\{1,\dots,n\}}$ thành $A_{I,\cdot}$, và $A_{\{1,\dots,m\},J}$ thành $A_{\cdot,J}$.

Tương tự, ta ký hiệu $A^{I,J}$ là ma trận con thu được bằng cách xóa khỏi A mọi hàng trong I và mọi cột trong J .

Với một ma trận vuông $n \times n$ A , định thức của nó được ký hiệu là $\det A$, và

$$\det A = \sum_{\pi \in \Sigma_n} (-1)^{\text{sgn } \pi} \prod_{k=1}^n A_{k,\pi(k)}.$$

Trong đó Σ_n biểu diễn tập mọi hoán vị của $\{1, \dots, n\}$, còn $\text{sgn } \pi$ biểu diễn dấu của hoán vị π . Nếu số cặp nghịch thế của dãy $\{\pi(1), \dots, \pi(n)\}$ là x , thì $\text{sgn } \pi = (-1)^x$.

Cho m vector n chiều $\{v_1, \dots, v_m\}$. Nếu tồn tại $\lambda_1, \lambda_2, \dots, \lambda_m$ sao cho

$$v = \sum_{i=1}^m \lambda_i v_i,$$

thì gọi v là một tổ hợp tuyến tính của $\{v_1, \dots, v_m\}$. Nếu không vector nào trong m vector này có thể được biểu diễn thành tổ hợp tuyến tính của các vector còn lại, ta nói m vector này độc lập tuyến tính; ngược lại, chúng phụ thuộc tuyến tính.

Với một ma trận A , nếu xem mỗi hàng của nó là một vector, thì số lượng cực đại các vector độc lập tuyến tính có thể chọn ra được gọi là **hạng hàng**; nếu xem mỗi cột là một vector, số lượng cực đại các

vector cột độc lập tuyến tính được gọi là **hạng cột**. Hạng hàng và hạng cột của một ma trận luôn bằng nhau, nên ta gọi chung là **hạng** của ma trận A , ký hiệu $\text{rank } A$.

Cuối cùng, ta nêu một kết luận mà không chứng minh.

Định lý 1.1. Bốn bài toán sau có cùng độ phức tạp:

1. tính định thức của ma trận;
2. nhân hai ma trận;
3. tìm ma trận nghịch đảo;
4. khử Gauss trên ma trận, tức phân rã LU.

Ta đặt độ phức tạp của cả bốn bài toán này là $\mathcal{O}(n^\omega)^4$.

4.2 Sự tồn tại của matching hoàn hảo

4.2.1 Ma trận Tutte

Định nghĩa 2.1 (Ma trận Tutte). Với một đồ thị vô hướng $G = (V, E)$, định nghĩa ma trận Tutte của G là một ma trận $n \times n$ $\tilde{A}(G)$, trong đó

$$\tilde{A}(G)_{i,j} = \begin{cases} x_{i,j}, & \text{nếu } v_i v_j \in E \text{ và } i < j, \\ -x_{j,i}, & \text{nếu } v_i v_j \in E \text{ và } i > j, \\ 0, & \text{trong các trường hợp khác.} \end{cases}$$

Ở đây, $x_{i,j}$ là một biến duy nhất gắn với cạnh $v_i v_j$.

Ta xét ví dụ sau.



Hình 1: Đồ thị G và ma trận Tutte của nó

Nửa trái của Hình 1 cho một đồ thị G , còn nửa phải cho ma trận Tutte $\tilde{A}(G)$ tương ứng với G . Có thể thấy mỗi cạnh của G đều tương ứng với một biến, nên $\tilde{A}(G)$ dùng tổng cộng $|E|$ biến.

4.2.2 Định lý Tutte

Ma trận Tutte có liên hệ mật thiết với sự tồn tại của matching hoàn hảo trong đồ thị. Thật vậy, ta có định lý sau.

Định lý 2.1 (Tutte). Đồ thị G có matching hoàn hảo khi và chỉ khi $\det \tilde{A}(G) \neq 0$.

Để chứng minh định lý này, trước hết ta cần dùng khái niệm “phủ chu trình chẵn” để đặc trưng một đồ thị có matching hoàn hảo.

⁴Trong học thuật, người ta thường cho rằng $2 \leq \omega < 2.373$.

Định nghĩa 2.2. Một **phủ chu trình** của đồ thị $G = (V, E)$ là việc dùng một số chu trình để phủ mọi đỉnh của G , sao cho mỗi đỉnh của G nằm đúng trong một chu trình.

Nói một cách hình thức, mỗi phủ chu trình của G tương ứng với một hoán vị π từ 1 đến n , thỏa mãn $v_i v_{\pi(i)} \in E$.

Nếu mọi chu trình trong phủ chu trình này đều có độ dài chẵn, ta gọi nó là một **phủ chu trình chẵn**; ngược lại, ta gọi nó là một **phủ chu trình lẻ**.

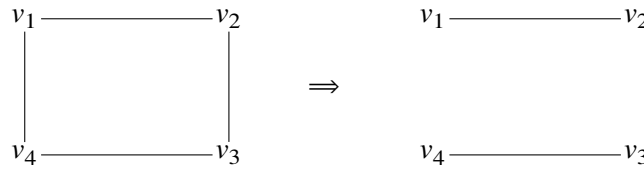
Chú ý rằng Định nghĩa 2.2 không yêu cầu các cạnh $v_i v_{\pi(i)}$ đôi một khác nhau, nghĩa là trong một phủ chu trình có thể xuất hiện cạnh lặp.

Bổ đề 2.2. Đồ thị $G = (V, E)$ có matching hoàn hảo khi và chỉ khi G có một phủ chu trình chẵn.

Chứng minh. Nếu G có một matching hoàn hảo, ta trực tiếp dùng chu trình độ dài hai do mỗi cặp đỉnh trong matching tạo thành để phủ G .

Nếu G có một phủ chu trình chẵn, thì trong mỗi chu trình chẵn, lấy cách một cạnh một cạnh, ta sẽ thu được một matching hoàn hảo của G . ■

Hình 2 dưới đây minh họa cách thu được một matching hoàn hảo từ một phủ chu trình chẵn của G .



Hình 2: Phủ chu trình chẵn và matching hoàn hảo

Có Bổ đề 2.2, ta có thể chứng minh định lý Tutte, tức Định lý 2.1.

Chứng minh định lý Tutte. Ta có

$$\det \tilde{A}(G) = \sum_{\pi \in \Sigma_n} (-1)^{\text{sgn}(\pi)} \prod_{k=1}^n \tilde{A}(G)_{k, \pi(k)}. \quad (2.1)$$

Các hạng tử khác 0 ở vế phải của đẳng thức có dạng

$$(-1)^{\text{sgn}(\pi)} \prod_{k=1}^n \pm x_{k, \pi(k)},$$

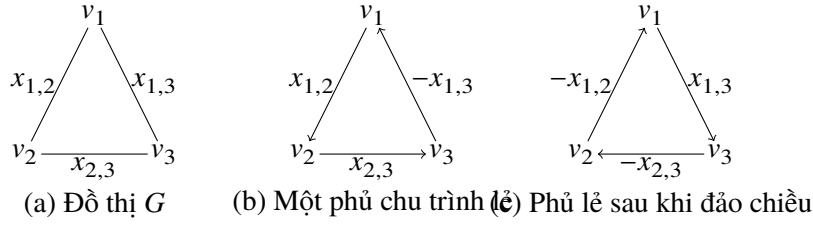
trong đó $v_1 v_{\pi(1)}, \dots, v_n v_{\pi(n)}$ là các cạnh của G . Những cạnh này tạo thành một phủ chu trình của G , tương ứng với phân rã thành chu trình của π .

Tiếp theo, nếu ta đảo chiều một chu trình bất kỳ có độ dài lớn hơn 2 trong π , ta sẽ thu được cùng một phủ chu trình. Bây giờ xét dấu của nó thay đổi ra sao. Gọi π' là hoán vị thu được sau khi đảo chiều một chu trình trong π . Việc đảo chiều một chu trình không làm đổi dấu của hoán vị π , tức $\text{sgn}(\pi) = \text{sgn}(\pi')$, nhưng làm đổi dấu mọi biến $x_{i, \pi(i)}$ trên chu trình đó.

Vì vậy, nếu ta đảo chiều một chu trình chẵn trong π , dấu của hạng tử thu được giống với dấu của hạng tử tương ứng với π ; nếu không, dấu ngược lại.

Do đó mọi phủ chu trình chứa chu trình lẻ đều bị triệt tiêu từng cặp một bởi một hạng tử dương và một hạng tử âm. Vì $\det \tilde{A}(G) \neq 0$, điều này có nghĩa là đồ thị G tồn tại một phủ chu trình chẵn, nên G có matching hoàn hảo. ■

Để hiểu trực quan hơn chứng minh này, ta lấy một chu trình ba đỉnh làm ví dụ để giải thích vì sao phủ chu trình lẻ sẽ không xuất hiện trong $\det \tilde{A}(G)$.



Hình 3: Chu trình ba đỉnh G

Ta lấy một phủ chu trình lẻ của G để phân tích. Như Hình 3(b), giả sử phủ chu trình ta chọn tương ứng với hoán vị $\pi = (2\ 3\ 1)$. Khi đó đóng góp của hạng tử này vào $\det \tilde{A}(G)$ là $\text{sgn}(\pi)x_{1,2}x_{2,3}(-x_{1,3})$. Bây giờ đảo chiều π , ta thu được Hình 3(c), tương ứng với hoán vị $\pi' = \pi^{-1} = (3\ 1\ 2)$, và đóng góp của nó vào $\det \tilde{A}(G)$ là $\text{sgn}(\pi')(-x_{1,2})(-x_{2,3})x_{1,3}$.

Rõ ràng $\text{sgn}(\pi) = \text{sgn}(\pi')$; đồng thời có thể thấy đóng góp của π' chính là đóng góp của π sau khi nhân biến tương ứng với mỗi cạnh trên chu trình với -1 . Vì vậy hai hạng tử này trái dấu nhau, và tổng đóng góp của chúng là 0.

4.2.3 Ngẫu nhiên hóa

Định lý Tutte, tức Định lý 2.1, cho ta một thuật toán quyết định rất tốt để kiểm tra đồ thị G có matching hoàn hảo hay không: chỉ cần xét $\det \tilde{A}(G)$ có bằng không hay không. Nhưng cách làm này có một vấn đề đáng kể: mỗi hạng tử của ma trận Tutte đều là một biến, và tổng số biến lên tới $|E|$, nên việc tính trực tiếp định thức rất khó, thậm chí cần thời gian mũ.

Tuy nhiên, ta không nhất thiết phải tính ra $\det \tilde{A}(G)$; việc ta cần làm chỉ là phán đoán $\det \tilde{A}(G)$ có đồng nhất bằng 0 hay không.

Quyết định một đa thức có đồng nhất bằng 0 hay không là một bài toán kinh điển. Giả sử hiện tại ta có một đa thức n biến $P(x_1, x_2, \dots, x_n)$, và muốn quyết định nó có đồng nhất bằng 0 hay không. Ta nên làm gì?

Trước hết có thể quan sát rằng nếu $P(x_1, x_2, \dots, x_n) = 0$, thì bất kể ta thay giá trị nào vào đa thức P , kết quả thu được chắc chắn đều là 0. Vậy liệu ta có thể chọn n số ngẫu nhiên $r_1, r_2, \dots, r_n$, rồi dựa vào việc $P(r_1, r_2, \dots, r_n)$ có bằng 0 hay không để suy ra $P(x_1, x_2, \dots, x_n)$ có đồng nhất bằng 0 hay không? Bổ đề Schwartz–Zippel dưới đây cho ta biết cách làm trông có vẻ không đáng tin này thực ra có xác suất cao cho kết quả đúng.

Bổ đề 2.3 (Schwartz, Zippel). Với một đa thức n biến bậc d $P(x_1, x_2, \dots, x_n)$ trên trường F , không đồng nhất bằng 0, giả sử $r_1, r_2, \dots, r_n$ là n số ngẫu nhiên trong F được chọn độc lập, khi đó

$$\Pr[P(r_1, r_2, \dots, r_n) = 0] \leq \frac{d}{|F|}.$$

Chứng minh. Ta thử chứng minh định lý này bằng quy nạp.

Xét trường hợp một biến: khi đó P có nhiều nhất d nghiệm, nên xác suất ta vừa đúng chọn vào một nghiệm không vượt quá $d/|F|$.

Bây giờ giả sử định lý đúng cho trường hợp $n - 1$ biến. Ta xét cách phân loại mọi hạng tử của P theo bậc của x_1 :

$$P(x_1, x_2, \dots, x_n) = \sum_{i=0}^d x_1^i P_i(x_2, \dots, x_n).$$

Vì $P \neq 0$, tồn tại một i sao cho $P_i \neq 0$. Xét i lớn nhất thỏa điều kiện đó, khi ấy $\deg P_i \leq d - i$.

Theo giả thiết quy nạp,

$$\Pr[P_i(r_2, \dots, r_n) = 0] \leq \frac{d-i}{|F|}.$$

Nếu $P_i(r_2, \dots, r_n) \neq 0$, thì $P(x_1, r_2, \dots, r_n)$ là một đa thức một biến bậc i , nên

$$\Pr[P(r_1, r_2, \dots, r_n) = 0 \mid P_i(r_2, \dots, r_n) \neq 0] \leq \frac{i}{|F|}.$$

Đặt sự kiện $P(r_1, r_2, \dots, r_n) = 0$ là A , và sự kiện $P_i(r_2, \dots, r_n) = 0$ là B . Khi đó

$$\begin{aligned} \Pr[A] &= \Pr[A \cap B] + \Pr[A \cap B^c] \\ &= \Pr[B] \Pr[A \mid B] + \Pr[B^c] \Pr[A \mid B^c] \\ &\leq \Pr[B] + \Pr[A \mid B^c] \\ &\leq \frac{d-i}{|F|} + \frac{i}{|F|} = \frac{d}{|F|}. \end{aligned}$$

■

Ta biết rằng với một đồ thị G cho trước, bậc của đa thức $\det \tilde{A}(G)$ là cấp $\mathcal{O}(n)$. Vì vậy ta có thể chọn một số nguyên tố p cấp n^2 , rồi trên trường modulo p , tức F_p , gán ngẫu nhiên một giá trị cho biến $x_{i,j}$ tương ứng với mỗi cạnh $v_i v_j$. Sau đó ta kiểm tra⁵ $\det \tilde{A}(G)$ có bằng 0 hay không để phán đoán G có matching hoàn hảo hay không. Theo bổ đề Schwartz–Zippel, xác suất phán đoán sai không vượt quá $\frac{n}{p} = \mathcal{O}(1/n)$; xác suất này chấp nhận được đối với chúng ta. Hơn nữa, ta có thể giảm xác suất sai bằng cách điều chỉnh kích thước của p , hoặc ngẫu nhiên nhiều lần.

Từ đó, ta có thuật toán quyết định sau.

Thuật toán 1 Lovasz’s testing algorithm

```

1: if  $\det \tilde{A}(G) \neq 0$  then
2:   print “YES”
3: else
4:   print “NO”
5: end if
```

Định lý 2.4. Độ phức tạp thời gian của Thuật toán 1 là $\mathcal{O}(n^\omega)$.

4.3 Xây dựng phương án matching hoàn hảo

4.3.1 Một thuật toán vét cạn

Theo Thuật toán 1, hiện tại ta đã có thể quyết định đồ thị G có matching hoàn hảo hay không. Từ đó ta lập tức thu được một thuật toán vét cạn để xây dựng matching: duyệt từng cạnh, xóa hai đầu mút của nó, rồi kiểm tra đồ thị còn lại có matching hoàn hảo hay không.

⁵Từ đây trở đi, khi cần tính định thức, ma trận nghịch đảo, hoặc khử ma trận $\tilde{A}(G)$, ta đều hiểu là sau khi đã gán ngẫu nhiên các biến thì thực hiện phép toán tương ứng trên F_p .

Thuật toán 2 A naive matching algorithm

```

1:  $M \leftarrow \emptyset$ 
2: for  $i \leftarrow 1$  to  $n$  do
3:   for  $j \leftarrow 1$  to  $n$  do
4:     if  $v_i v_j \in E(G)$  and  $\det \tilde{A}(G - \{v_i, v_j\}) \neq 0$  then
5:        $G \leftarrow G - \{v_i, v_j\}$ 
6:        $M \leftarrow M \cup \{v_i v_j\}$ 
7:     end if
8:   end for
9: end for
10: return  $M = 0$ 

```

Thuật toán 2 cần thực hiện $\mathcal{O}(n^2)$ vòng lặp; trong mỗi vòng lặp cần tính định thức của một ma trận vuông cấp $\mathcal{O}(n)$. Do đó:

Định lý 3.1. Độ phức tạp thời gian của Thuật toán 2 là $\mathcal{O}(n^{\omega+2})$.

4.3.2 Một số kiến thức đại số tuyến tính

Nút thắt của Thuật toán 2 nằm ở việc cần phán đoán mỗi cạnh có thể nằm trong matching hoàn hảo hay không. Nếu ta có thể tăng tốc quá trình phán đoán này, ta sẽ giảm được độ phức tạp của thuật toán.

Để đạt mục tiêu đó, trước hết ta cần biết một số kiến thức đại số tuyến tính.

Định nghĩa 3.1. Phần phụ của ma trận A theo hàng i và cột j , ký hiệu $M_{i,j}$, là $\det A^{(i), (j)}$.

Định nghĩa 3.2. Đại số phụ của ma trận A theo hàng i và cột j , ký hiệu $C_{i,j}$, là $(-1)^{i+j} M_{i,j}$.

Ma trận các đại số phụ của A là một ma trận $n \times n$ C , thỏa mãn phần tử ở hàng i , cột j của nó là đại số phụ của A theo hàng i và cột j .

Định nghĩa 3.3. Ma trận phụ hợp $\text{adj } A$ của ma trận A là chuyển vị của ma trận các đại số phụ của A , tức $\text{adj } A = C^T$.

Định lý 3.2 (Khai triển Laplace). Giả sử A là một ma trận $n \times n$. Khi đó với một hàng bất kỳ $i \in \{1, \dots, n\}$, ta có

$$\det A = \sum_{j=1}^n A_{i,j} C_{i,j} = \sum_{j=1}^n A_{i,j} (\text{adj } A)_{j,i}.$$

Tương tự, với một cột bất kỳ $j \in \{1, \dots, n\}$, ta có

$$\det A = \sum_{i=1}^n A_{i,j} (\text{adj } A)_{j,i}.$$

Định lý 3.3. Nếu ma trận A khả nghịch, thì

$$A^{-1} = \text{adj } A / \det A.$$

Chứng minh. Theo Định nghĩa 3.2 và Định nghĩa 3.3, hệ số ở hàng i , cột i của $A \times \text{adj } A$ là $\sum_{j=1}^n A_{i,j} C_{i,j}$. Theo Định lý 3.2, hệ số này bằng $\det A$.

Nếu $i \neq j$, thì hệ số ở hàng i , cột j của $A \times \text{adj } A$ là $\sum_{k=1}^n A_{i,k} C_{j,k}$. Thực ra, điều này tương đương với việc thay các phần tử hàng j của A bằng các phần tử hàng i , rồi lấy định thức. Vì có hai hàng giống nhau, định thức bằng 0.

Vì vậy ta có

$$A \times \text{adj } A = \text{adj } A \times A = \det A \times I_n,$$

tức là

$$A^{-1} = \text{adj } A / \det A.$$

■

4.3.3 Tính chất của ma trận phản đối xứng

Trước hết ta định nghĩa khái niệm ma trận phản đối xứng.

Định nghĩa 3.4. Với một ma trận $n \times n$ A , nếu $A^T = -A$, tức $A_{i,j} = -A_{j,i}$, thì ta gọi A là một ma trận phản đối xứng.

Ma trận Tutte trong Định nghĩa 2.1 chính là một ma trận phản đối xứng. Có thể thấy các thuật toán phía trước của ta đều xoay quanh việc tính định thức của ma trận Tutte; đây chính là lý do ta nghiên cứu tính chất của ma trận phản đối xứng.

Bổ đề 3.4. Với một ma trận phản đối xứng $n \times n$ A , nếu n lẻ thì $\det A = 0$.

Chứng minh. Từ kiến thức về định thức, ta biết $\det A = \det A^T$. Lại vì $A^T = -A$, nên $\det A = \det(-A) = (-1)^n \det A$.

Khi n lẻ, $(-1)^n = -1$, tức $\det A = -\det A$, do đó $\det A = 0$.

■

Định lý 3.5. Với một ma trận phản đối xứng $n \times n$ A và một tập chỉ số hàng $I = \{i_1, i_2, \dots, i_k\}$, nếu $A_{i_1, \cdot}, A_{i_2, \cdot}, \dots, A_{i_k, \cdot}$ là một nhóm độc lập tuyến tính cực đại theo hàng của A , thì $\det A_{I,I} \neq 0$.

Chứng minh. Không mất tính tổng quát, giả sử $I = \{1, 2, \dots, k\}$.

Vì $A_{1, \cdot}, A_{2, \cdot}, \dots, A_{k, \cdot}$ là một nhóm độc lập tuyến tính cực đại theo hàng của A , từ tính phản đối xứng của A , ta có $A_{\cdot, 1}, A_{\cdot, 2}, \dots, A_{\cdot, k}$ là một nhóm độc lập tuyến tính cực đại theo cột của A .

Do đó với mọi $k < i \leq n$, $A_{\cdot, i}$ đều có thể được biểu diễn tuyến tính bởi $A_{\cdot, 1}, A_{\cdot, 2}, \dots, A_{\cdot, k}$. Vì vậy

$$\text{rank } A_{I,I} = \text{rank } A_{I, \cdot} = k,$$

tức $\det A_{I,I} \neq 0$.

■

Hệ quả 3.6. Với một ma trận phản đối xứng A , tồn tại một tập chỉ số hàng $I = \{i_1, i_2, \dots, i_k\}$ sao cho

$$\text{rank } A = \text{rank } A_{I,I} = k.$$

Định lý 3.7. Hạng của một ma trận phản đối xứng là số chẵn.

Chứng minh. Theo Hệ quả 3.6, một ma trận phản đối xứng tồn tại một định thức con chính đầy hạng có hạng bằng hạng của chính nó. Vì định thức con chính của ma trận phản đối xứng cũng là phản đối xứng, kết hợp với Bổ đề 3.4 ta biết hạng của định thức con chính này là số chẵn. Do đó hạng của một ma trận phản đối xứng là số chẵn. ■

Bổ đề 3.8. Với một ma trận phản đối xứng khả nghịch $n \times n$ A , và hai chỉ số bất kỳ $i, j \in \{1, \dots, n\}, i \neq j$, ta có

$$\det A^{(i),\{j\}} \neq 0 \iff \det A^{\{i,j\},\{i,j\}} \neq 0.$$

Chứng minh. Nếu $\det A^{(i),\{j\}} \neq 0$, thì $\text{rank} A^{(i),\{j\}} = n - 1$. Vì $A^{\{i,j\},\{i,j\}}$ thu được từ $A^{(i),\{j\}}$ bằng cách xóa thêm một hàng và một cột, nên $\text{rank} A^{\{i,j\},\{i,j\}} \geq n - 3$. Lại vì $A^{\{i,j\},\{i,j\}}$ là ma trận phản đối xứng, theo Định lý 3.7, hạng của nó là số chẵn, nên $\text{rank} A^{\{i,j\},\{i,j\}}$ nhất định bằng $n - 2$. Do đó $\det A^{\{i,j\},\{i,j\}} \neq 0$.

Ngược lại, nếu $\det A^{\{i,j\},\{i,j\}} \neq 0$, thì $\text{rank} A^{\{i,j\},\{i,j\}} = n - 2$; đồng thời $\text{rank} A^{(i),\{j\}} = n - 2$. Theo Định lý 3.7,

$$\text{rank} A^{(i),\{i,j\}} = \text{rank} A^{(i),\{i\}} = n - 2.$$

Điều này cho thấy cột thứ j của $A^{(i),\emptyset}$ là tổ hợp tuyến tính của các cột khác. Vì vậy $\text{rank} A^{(i),\{j\}} = \text{rank} A^{(i),\emptyset} = n - 1$, suy ra $\det A^{(i),\{j\}} \neq 0$. ■

4.3.4 Thuật toán Rabin–Vazirani

Bây giờ ta thử thu được một thuật toán xây dựng phương án matching tốt hơn Thuật toán 2.

Định lý 3.9 (Rabin, Vazirani). Giả sử $G = (V, E)$ là một đồ thị có matching hoàn hảo, và $\tilde{A} = \tilde{A}(G)$ là ma trận Tutte tương ứng của G . Khi đó

$$(\tilde{A}^{-1})_{i,j} \neq 0 \iff G - \{v_i, v_j\} \text{ có matching hoàn hảo.}$$

Chứng minh. Định lý này có thể được suy ra từ Định lý 3.3, Bổ đề 3.8 và định lý Tutte, tức Định lý 2.1. ■

Dựa trên Định lý 3.9, ta thu được thuật toán sau.

Thuật toán 3 Matching algorithm of Rabin and Vazirani

```

1:  $M \leftarrow \emptyset$ 
2: for  $i \leftarrow 1$  to  $n$  do
3:   if  $v_i$  is not yet matched then
4:     compute  $\tilde{A}(G)^{-1}$ 
5:     find  $j$ , such that  $v_i v_j \in E(G)$  and  $(\tilde{A}(G)^{-1})_{j,i} \neq 0$ 
6:      $G \leftarrow G - \{v_i, v_j\}$ 
7:      $M \leftarrow M \cup \{v_i v_j\}$ 
8:   end if
9: end for
10: return  $M = 0$ 
```

Thuật toán 3 cần thực hiện tổng cộng $\mathcal{O}(n)$ vòng lặp; trong mỗi vòng lặp cần tìm nghịch đảo của một ma trận vuông cấp $\mathcal{O}(n)$. Do đó:

Định lý 3.10. Độ phức tạp của Thuật toán 3 là $\mathcal{O}(n^{\omega+1})$.

4.3.5 Phương pháp xây dựng dựa trên khử Gauss

Nút thắt của Thuật toán 3 nằm ở việc tính nghịch đảo của ma trận Tutte $\tilde{A}(G)$. Thực ra, trong mỗi vòng lặp ta chỉ xóa hai hàng và hai cột của $\tilde{A}(G)$, nhưng lại cần tính lại ma trận nghịch đảo từ đầu. Nếu mỗi lần không cần tính lại nghịch đảo từ đầu, mà có thể dùng một phương pháp hiệu quả để duy trì ma trận nghịch đảo, ta sẽ giảm được độ phức tạp của thuật toán này.

Thuật toán của ta dựa trên định lý sau.

Định lý 3.11. Nếu

$$A = \begin{pmatrix} a_{1,1} & v^T \\ u & B \end{pmatrix}, \quad A^{-1} = \begin{pmatrix} \hat{a}_{1,1} & \hat{v}^T \\ \hat{u} & \hat{B} \end{pmatrix},$$

trong đó $\hat{a}_{1,1} \neq 0$, thì

$$B^{-1} = \hat{B} - \hat{u}\hat{v}^T/\hat{a}_{1,1}.$$

Chứng minh. Vì $AA^{-1} = I$, ta có

$$\begin{pmatrix} a_{1,1}\hat{a}_{1,1} + v^T\hat{u} & a_{1,1}\hat{v}^T + v^T\hat{B} \\ u\hat{a}_{1,1} + B\hat{u} & u\hat{v}^T + B\hat{B} \end{pmatrix} = \begin{pmatrix} 1 & 0 \\ 0 & I_{n-1} \end{pmatrix}.$$

Từ đó,

$$\begin{aligned} B(\hat{B} - \hat{u}\hat{v}^T/\hat{a}_{1,1}) &= I_{n-1} - u\hat{v}^T - B\hat{u}\hat{v}^T/\hat{a}_{1,1} \\ &= I_{n-1} - u\hat{v}^T + u\hat{a}_{1,1}\hat{v}^T/\hat{a}_{1,1} \\ &= I_{n-1} - u\hat{v}^T + u\hat{v}^T \\ &= I_{n-1}. \end{aligned}$$

■

Chú ý rằng trong định lý trên, thao tác sửa $\hat{B}$ đúng là một thao tác khử trong khử Gauss. Trong định lý, ta chứng minh với ví dụ khử hàng thứ nhất và cột thứ nhất.

Là một ứng dụng trực tiếp của Định lý 3.11, ta thu được thuật toán sau.

Thuật toán 4 Simple matching algorithm

- 1: $M \leftarrow \emptyset$
- 2: compute $\tilde{A}(G)^{-1}$
- 3: **for** $i \leftarrow 1$ **to** n **do**
- 4: **if** the i -th row is not yet eliminated **then**
- 5: find j such that $v_i v_j \in E(G)$ and $(\tilde{A}(G)^{-1})_{j,i} \neq 0$
- 6: $G \leftarrow G - \{v_i, v_j\}$
- 7: $M \leftarrow M \cup \{v_i v_j\}$
- 8: update $\tilde{A}(G)^{-1}$ by eliminating the i -th row and the j -th column and then the j -th row and the i -th column
- 9: **end if**
- 10: **end for**
- 11: **return** $M = 0$

Thuật toán 4 cần lặp $\mathcal{O}(n)$ lần; trong mỗi vòng lặp cần thực hiện hai thao tác khử. Độ phức tạp thời gian của một thao tác khử là $\mathcal{O}(n^2)$. Vì vậy:

Định lý 3.12. Độ phức tạp thời gian của Thuật toán 4 là $\mathcal{O}(n^3)$.

4.4 Matching cực đại

4.4.1 Kích thước của matching cực đại

Đến đây, mọi thuật toán của ta đều dùng để giải matching hoàn hảo, nhưng trong ứng dụng thực tế, bài toán ta cần giải thường là matching cực đại.

Tương tự ý tưởng giải bài toán matching hoàn hảo phía trước, trước hết ta bắt đầu từ bài toán đơn giản nhất: với một đồ thị cho trước $G = (V, E)$, làm thế nào tìm được kích thước matching cực đại $\nu(G)$ của G ? Với bài toán này, ta có định lý sau.

Định lý 4.1. Kích thước matching cực đại $\nu(G)$ của đồ thị G là

$$\frac{1}{2} \text{rank } \tilde{A}(G).$$

Chứng minh. Đặt $k = \text{rank } \tilde{A}(G)$. Theo Hệ quả 3.6, ta có thể chọn một tập chỉ số hàng $I = \{i_1, i_2, \dots, i_k\}$ của $\tilde{A}(G)$ sao cho

$$\text{rank } \tilde{A}(G) = \text{rank } \tilde{A}(G)_{I,I} = k.$$

Đặt $U = \{v_{i_1}, v_{i_2}, \dots, v_{i_k}\}$. Khi đó $\tilde{A}(G)_{I,I} = \tilde{A}(G[U])$. Vì $\text{rank } \tilde{A}(G[U]) = |U|$, đồ thị $G[U]$ tồn tại matching hoàn hảo; điều này cho thấy $\nu(G) \geq k/2$.

Nếu $\nu(G) > k/2$, ta xét lấy ra mọi đỉnh nằm trong một matching cực đại. Khi đó nhất định tồn tại một tập chỉ số hàng I sao cho $|I| > k$ và $\det \tilde{A}(G)_{I,I} \neq 0$. Điều này có nghĩa là hạng của một ma trận con của $\tilde{A}(G)$ lớn hơn hạng của chính nó, hiển nhiên là không thể.

Tổng hợp lại, $\nu(G) = k/2$. ■

4.4.2 Chuyển hóa thứ nhất

Tiếp theo ta thử chuyển bài toán matching cực đại về bài toán matching hoàn hảo quen thuộc. Nếu với một đồ thị G , ta đã tìm được kích thước matching cực đại $\nu(G)$ của nó, thì ta có thể thêm $n - 2\nu(G)$ đỉnh mới vào G , rồi nối cạnh từ mỗi đỉnh mới đến từng đỉnh trong n đỉnh ban đầu. Đồ thị mới thu được khi đó là một đồ thị có matching hoàn hảo, và matching hoàn hảo của đồ thị mới tương ứng với matching cực đại của đồ thị ban đầu.

Như vậy ta đã giải được bài toán matching cực đại. Chuyển hóa này có tính tổng quát rất tốt, nhưng có một nhược điểm: trong trường hợp xấu nhất, ta có thể phải thêm n đỉnh mới, làm kích thước dữ liệu, tức số đỉnh, tăng gấp đôi, nên hằng số của thuật toán sẽ tăng đáng kể.

4.4.3 Chuyển hóa thứ hai

Ta xét một cách làm khác, cũng chuyển bài toán matching cực đại về bài toán matching hoàn hảo, nhưng lần này thực hiện bằng cách xóa đỉnh. Với một đồ thị $G = (V, E)$ và một matching cực đại M của nó, xét tập đỉnh U gồm mọi đỉnh nằm trong matching M . Rõ ràng $G[U]$ có matching hoàn hảo, và matching hoàn hảo của $G[U]$ chính là matching cực đại của G . Ý tưởng của ta là tìm một tập đỉnh U như vậy, rồi áp dụng thuật toán giải matching hoàn hảo phía trước cho $G[U]$.

Thực ra, quá trình chứng minh Định lý 4.1 đã chỉ đường cho ta. Từ Bổ đề 3.5 và Hệ quả 3.6, ta chỉ cần tìm một nhóm độc lập tuyến tính cực đại theo hàng của $\tilde{A}(G)$. Điều này có thể được tìm bằng một lượt khử Gauss. Vì vậy, ta có thể chuyển bài toán matching cực đại về matching hoàn hảo trong độ phức tạp $\mathcal{O}(n^\omega)$, và nó không làm tăng hằng số của thuật toán.

4.5 Ứng dụng

Thuật toán được trình bày trong bài là thuật toán tổng quát để giải bài toán matching cực đại trên đồ thị tổng quát, nên tất nhiên nó có thể giải mọi bài cần dùng matching tổng quát.

Ngoài ra, nó còn có một số ứng dụng khá đặc sắc. Dưới đây ta xét hai ví dụ.

4.5.1 Vertex Geography⁶

Mô tả bài toán. Alice và Bob chơi một trò chơi trên một đồ thị vô hướng $G = (V, E)$. Hai người luân phiên thao tác, Alice đi trước.

Ban đầu có một quân cờ đặt trên một đỉnh nào đó. Sau đó, trong mỗi lượt, người chơi có thể di chuyển quân cờ này sang một đỉnh kề và chưa từng đi tới trước đó, tính cả điểm xuất phát. Người không thể thao tác sẽ thua. Hỏi những đỉnh nào là đỉnh thắng cho người đi trước?

Giới thiệu thuật toán. Trước hết ta có một kết luận: một đỉnh v_i là trạng thái thắng khi và chỉ khi nó nhất định nằm trong matching cực đại của đồ thị G .⁷

Vì vậy bài toán chuyển thành phán đoán một đỉnh nào đó có nhất định nằm trong matching cực đại của G hay không. Ta làm theo Mục 4.2: thêm $k = n - 2\nu(G)$ đỉnh mới và nối các cạnh tương ứng. Gọi đồ thị mới thu được là G' , các đỉnh mới có chỉ số lần lượt là $n+1, n+2, \dots, n+k$. Sau đó tùy ý chọn một đỉnh mới v_j ($n+1 \leq j \leq n+k$). Khi đó với mỗi đỉnh v_i của đồ thị ban đầu ($1 \leq i \leq n$), nếu $v_i v_j$ có thể xuất hiện trong một matching hoàn hảo của G' , thì v_i không nhất định nằm trong matching cực đại của G .

Theo Định lý 3.9, ta chỉ cần kiểm tra $(\tilde{A}(G')^{-1})_{i,j}$ có bằng không hay không.

Có thể thấy thuật toán matching được giới thiệu trong bài có ưu thế trong các bài toán kiểu “một đỉnh có nhất định nằm trong matching cực đại hay không” và “một cạnh có nhất định nằm trong matching cực đại hay không”. Lý do là thuật toán này tránh được việc phân tích đường tăng, mà chỉ cần kiểm tra một vị trí nào đó của ma trận nghịch đảo có bằng không hay không.

4.5.2 Matching trọng số cực đại

Mô tả bài toán. Matching trọng số cực đại là bài toán trong đó mỗi cạnh $v_i v_j$ của đồ thị G mang một trọng số $w_{i,j}$, và mục tiêu hiện tại của ta là tìm một matching M có tổng trọng số cạnh lớn nhất. Matching cực đại tương đương với matching trọng số cực đại khi mọi trọng số cạnh đều bằng 1.

Giới thiệu thuật toán. Ta có thể nối cạnh trọng số 0 giữa mọi cặp đỉnh; khi n lẻ, thêm một đỉnh mới và nối nó với mọi đỉnh. Vì vậy mỗi matching có trọng số của đồ thị ban đầu sẽ tương ứng với ít nhất một matching hoàn hảo có trọng số của đồ thị mới; và một matching hoàn hảo có trọng số của đồ thị mới tương ứng với một matching có trọng số của đồ thị ban đầu.

Bây giờ ta chuyển thành bài toán tìm matching hoàn hảo có trọng số của đồ thị mới G' . Ta thêm một biến y . Đặt

$$\tilde{A}'(G')_{i,j} = \tilde{A}(G')_{i,j} \times y^{w_{i,j}}.$$

Như vậy, điều ta cần tìm tương đương với bậc cao nhất của y trong $\det \tilde{A}'(G')$; giá trị này chia cho 2 chính là matching trọng số cực đại.

⁶Bài toán kinh điển này có tên tiếng Anh là Undirected Vertex Geography (UVG).

⁷Chứng minh kết luận này không phải trọng tâm của bài viết, nên được lược bỏ ở đây; độc giả quan tâm có thể tham khảo các tài liệu liên quan.

Trước hết, ta gán một giá trị ngẫu nhiên cho mọi $x_{i,j}$. Sau đó giả sử tổng trọng số của mọi cạnh không vượt quá W . Ta có thể lần lượt thay mọi giá trị từ 0 đến W vào y , tính một lần định thức, rồi dùng $W + 1$ kết quả này để nội suy một đa thức theo y . Khi đó bậc cao nhất của đa thức này là thứ ta cần tìm.

Độ phức tạp thời gian của thuật toán này là $\mathcal{O}(Wn^\omega + W^2)$; khi cả W và n đều nhỏ, nó có giá trị nhất định.

4.6 Tổng kết

Đến đây, ta đã có thể tìm kích thước matching cực đại trên đồ thị tổng quát trong độ phức tạp $\mathcal{O}(n^\omega)$, và xây dựng một phương án matching trong độ phức tạp $\mathcal{O}(n^3)$.

Thuật toán được giới thiệu trong bài vẫn còn tiềm năng rất lớn đang chờ chúng ta khám phá, và các ứng dụng được giới thiệu trong bài còn khá cơ bản. Vì vậy tôi hy vọng bài viết này có thể đóng vai trò gợi mở, mong rằng có độc giả sẽ mở rộng thuật toán trong bài, phát triển nó hơn nữa và thu được những ứng dụng thú vị hơn.

Lời cảm ơn

- Cảm ơn China Computer Federation đã cung cấp nền tảng học tập và trao đổi.
- Cảm ơn thầy Song Xinbo và thầy Zhuo Mingcong đã quan tâm và chỉ dẫn trong nhiều năm.
- Cảm ơn huấn luyện viên đội tuyển quốc gia Zhang Ruizhe và Yu Linyun đã chỉ dẫn.
- Cảm ơn bạn Zhou Zixin đã chia sẻ thuật toán này với tôi.
- Cảm ơn các bạn Gao Wenyuan và Wu Hongxun đã đọc duyệt bài viết này.
- Cảm ơn cha mẹ đã quan tâm và chăm sóc tôi.

Tài liệu tham khảo

- [1] Mucha M, Sankowski P. Maximum matchings via Gaussian elimination[C]//Foundations of Computer Science, 2004. Proceedings. 45th Annual IEEE Symposium on. IEEE, 2004: 248–255.
- [2] Ivan I, Virza M, Yuen H. Algebraic Algorithms for Matching[J]. 2011.
- [3] Cheung H Y. Algebraic algorithms in combinatorial optimization[D]. The Chinese University of Hong Kong, 2011.
- [4] Huang C C, Kavitha T. Efficient algorithms for maximum weight matchings in general graphs with small edge weights[C]//Proceedings of the twenty-third annual ACM-SIAM symposium on Discrete Algorithms. Society for Industrial and Applied Mathematics, 2012: 1400–1412.
- [5] Chen Yinbo. Bàn sơ về thuật toán matching trên đồ thị và ứng dụng của nó[C]//Tập luận văn ứng viên đội tuyển quốc gia Trung Quốc Olympic Tin học năm 2015. 2015.

Ghi chú về trích xuất

Nội dung bài đầu được trích bằng pdftotext từ trang vật lý 3 đến 13 của PDF, tương ứng trang bài viết 1 đến 11; trang vật lý 14 là bài kế tiếp. Không cần OCR. Hai dòng kết thúc vòng lặp trong mã giả của nguồn in là **end for** = 0; bản dịch giữ nguyên ký hiệu này như một dị biệt của nguồn.

Bài thứ hai được trích bằng pdftotext từ trang vật lý 14 đến 26 của PDF, tương ứng trang bài viết 12 đến 26. Các hình minh họa nhỏ được dựng lại bằng tikz; ký hiệu ma trận con xóa hàng/cột $A^{I,J}$ và các phân số trong chứng minh Định lý 4.1 được kiểm tra lại bằng trang render vì lớp văn bản của PDF làm mất chi tiết này.